



SESSION 7

ASYNCHRONOUS AND SYNCHRONOUS PROGRAMMING

Learning Objectives

In this session, students will learn to:

- Compare asynchronous and synchronous programming models
- Explain asynchronous programming model using `callback`, `Promise`, and `async/await` in Node.js
- Describe synchronous programming model in Node.js
- Explain the event loop architecture in Node.js

Asynchronous and synchronous programming models help developers create robust and event-driven applications. Asynchronous applications do not depend on the sequential approach of execution. Multiple tasks can be executed simultaneously. Synchronous applications follow sequential execution of tasks, one at a time.

This session will provide an overview of asynchronous and synchronous programming models. It will compare the features of both these models. The session will cover in detail the asynchronous programming model using `callback`, `Promise`, and `async/await` in Node.js. The session will also elaborate on the concept of synchronous programming model in Node.js.

The session will conclude with a detailed explanation of the event loop architecture in Node.js.

7.1 Introduction to Asynchronous Programming

A Node.js program can have multiple tasks for execution. The execution of the tasks depends on the adopted programming model. There are two generic programming models, asynchronous and synchronous.

An asynchronous programming model follows the non-blocking architecture, which allows a program to initiate multiple tasks at the same time. Each task runs concurrently without waiting for the other task to be completed. The application output provided to the user is faster. The application remains responsive throughout the execution cycle.

Consider an example of an online form application based on an asynchronous programming model. Figure 7.1 shows the visual representation of the simultaneous task execution in the application.

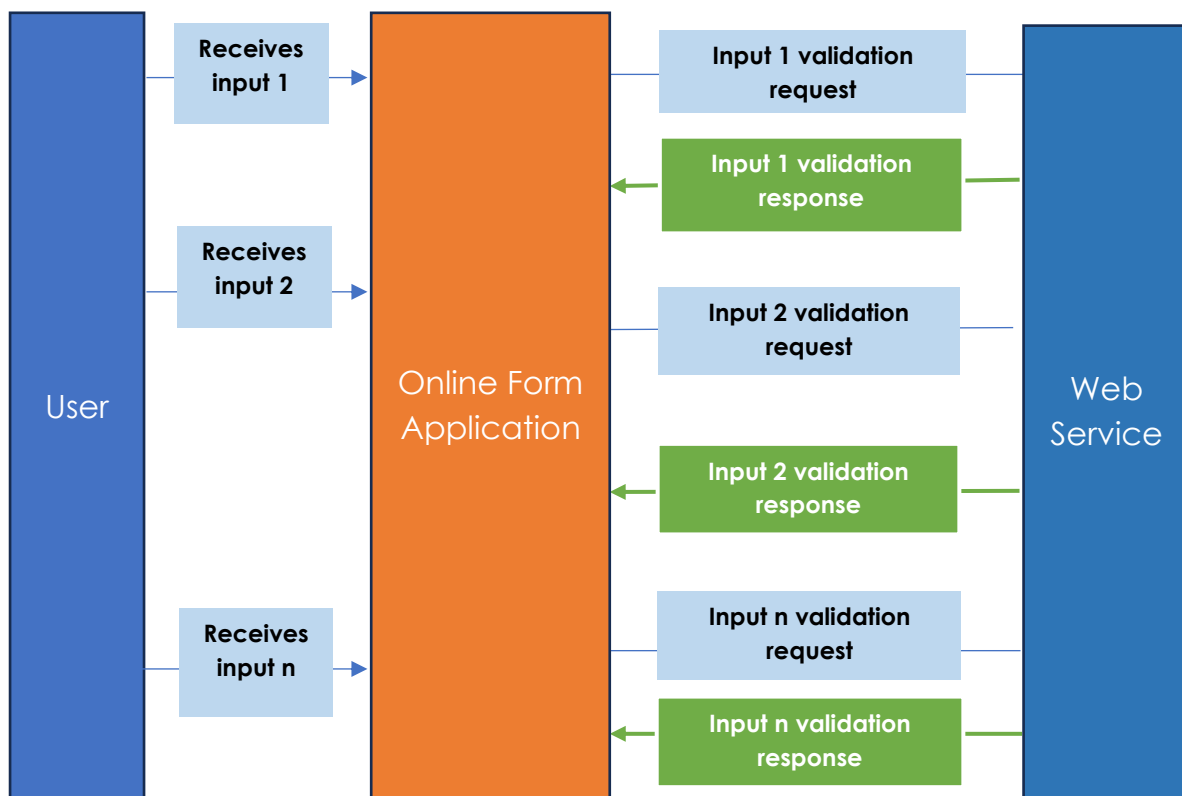
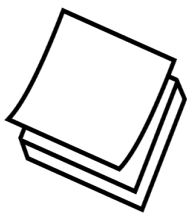


Figure 7.1: Asynchronous Task Execution

The form application continues to accept user inputs while it concurrently sends the received inputs to a Web service for validation. The Web service processes the validation request and immediately sends the response back to the application. This way, the application remains responsive throughout by continuously running the user interface in the foreground. Simultaneously, the application requests the Web service to perform the validation tasks in the background.

Asynchronous programming is used in both front-end and back-end applications. Different ways to implement asynchronous programming are by using the callback function, Promise object, `await/async` keywords, and so on.



Node.js is a single-threaded, asynchronous JavaScript runtime environment. The JavaScript code is executed in a single thread at a time. The asynchronous operations such as input request, network request, reading from a file, or writing to a file are executed simultaneously.

7.2 Introduction to Synchronous Programming

The synchronous programming model follows the blocking architecture that executes each task in a program in a sequential manner. The first task completes its execution and generates the output. The control then returns to the program for the execution of the second task. The second task is executed and the execution series goes on in a similar manner. If any one of the tasks consumes more time, the entire application may become unresponsive until the task is completed.

Consider the online form application example based on the synchronous programming model. Figure 7.2 shows the visual representation of the sequential task execution in the application.

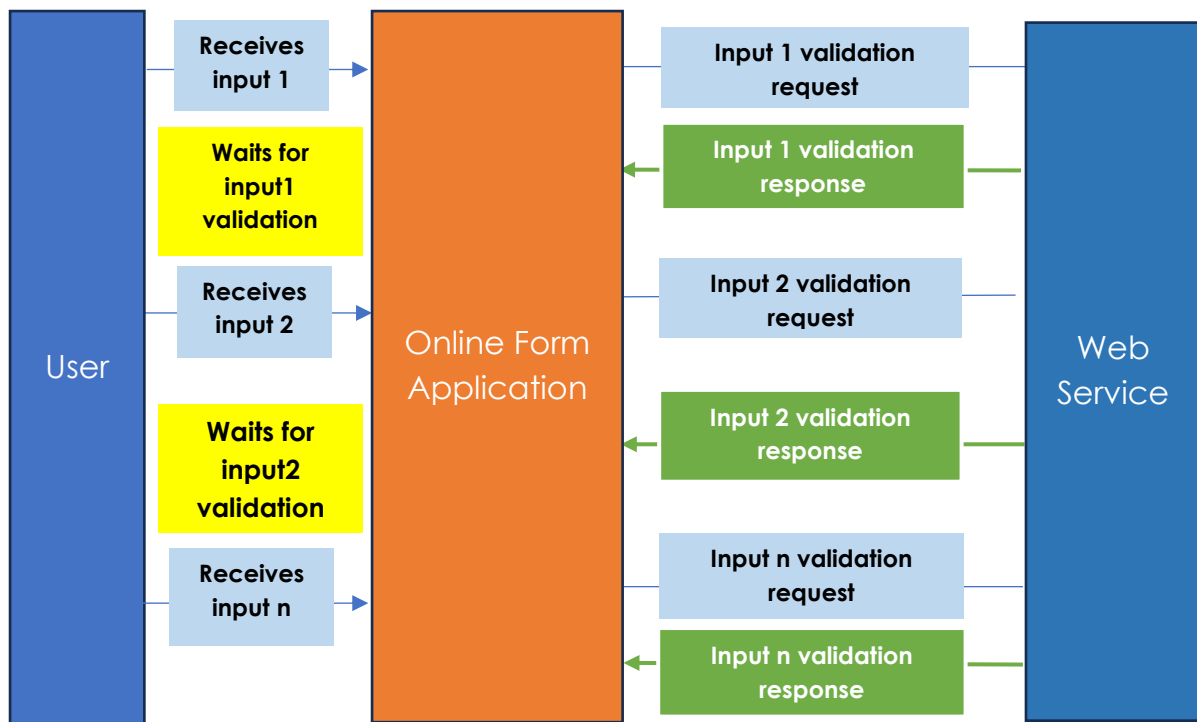
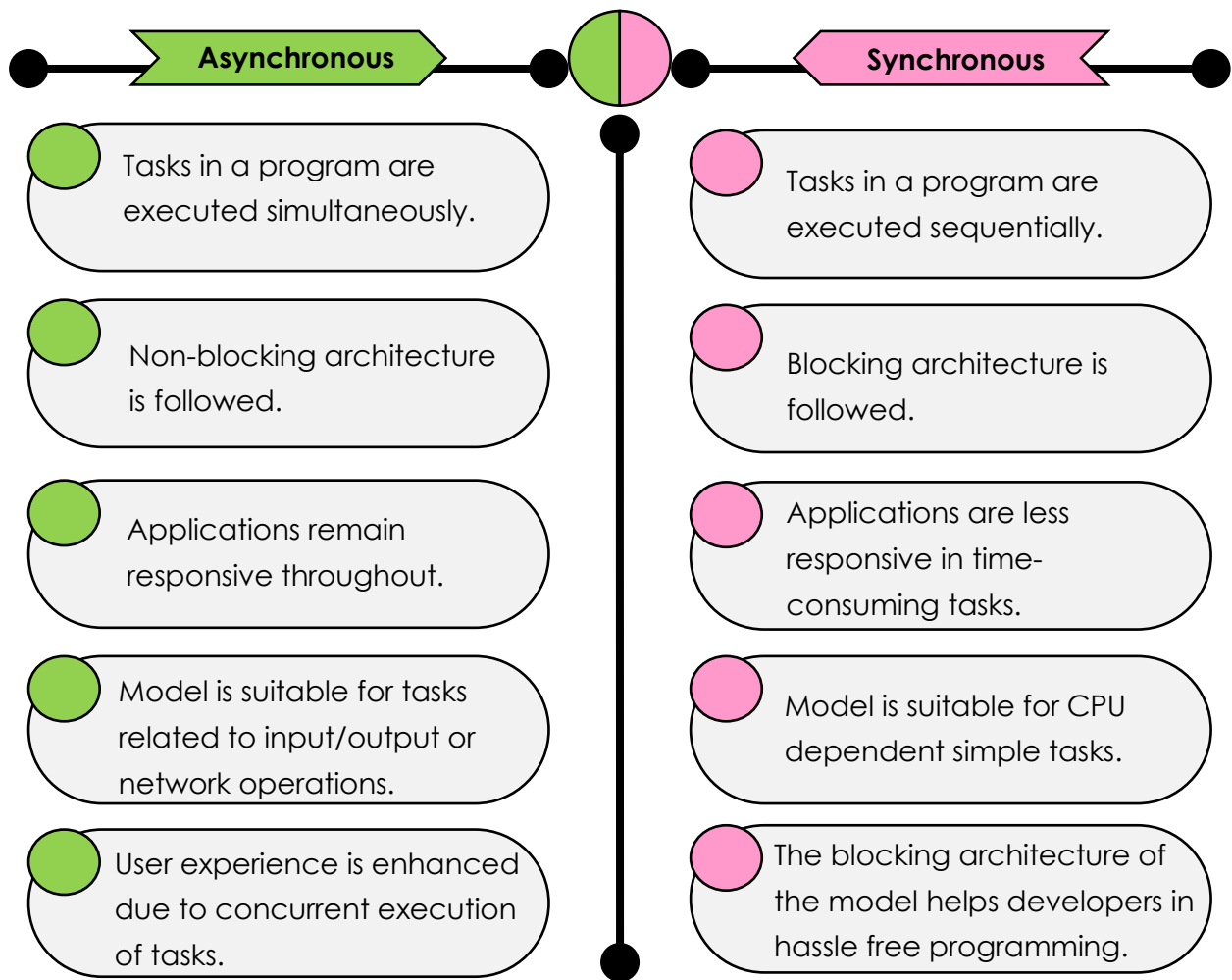


Figure 7.2: Synchronous Task Execution

The tasks in the form application are executed one at a time in a sequence. The application waits for the current input validation operation to complete before it moves on to receive the next input from the user. In this model, the order of executing the tasks is critical and important in the application.

7.3 Asynchronous vs. Synchronous Programming

As software developers, it is crucial to understand the difference between asynchronous and synchronous programming models.



7.4 Asynchronous Programming in Node.js

Node.js, preferably, follows the asynchronous programming model. Different ways to implement the asynchronous programming model in Node.js are as follows:

- Using the callback function
- With the `Promise` object
- With the help of `async` and `await` keywords

7.4.1 Asynchronous Programming in Node.js Using callback Function

A callback function is a function that is passed as an argument to another function. When the outer function finishes its task, it calls the callback function to handle the result of its operation. Thus, developers can use callback functions to implement asynchronous programming. Results of asynchronous operations can be handled without blocking the execution of the rest of the code.

Consider an example code in Code Snippet 1 where the text file reading operation is demonstrated using the callback function.

Perform the given steps:

1. Create a text file using VS Code or Notepad. Enter the contents in the sample text file:

```
Welcome to Node.js programming!
```

```
This session covers asynchronous and synchronous  
programming models.
```

2. Save the text file with the name `nodefile.txt` on the local system.
3. Type the code given in Code Snippet 1 in the editor.

Code Snippet 1:

```
const fs = require('fs');  
fs.readFile('nodefile.txt', 'utf8', (err, data) =>  
{  
  if (err) {  
    console.error(err);  
    return;  
  }  
  console.log(data);  
});  
console.log('Topics to be discussed in this session  
are:');  
console.log('1.Asynchronous Programming');  
console.log('2.Synchronous Programming');
```

The example code imports the `fs` module to perform the read operation on a text file. The `fs.readFile` function from the `fs` module reads the text file `nodefile.txt` asynchronously. The file reading operation takes place in the background while the remaining code gets executed simultaneously.

The last argument of the `fs.readFile` function is a callback function. The callback function is triggered when the background task of reading the file is completed. There are two arguments for the callback function.

The first argument `err` stores any potential error information if the task execution throws any error. The second argument `data` stores the contents of the text file. Once the `fs.readFile` function completes the

reading of the text file contents, the callback function displays the corresponding output on the console.

4. Save Code Snippet 1 with the file name `asyncprg.js`. Ensure to save the code in the same folder where the text file is saved.
5. Open the command prompt and navigate to the required folder.
6. To run the saved program, at the command prompt, type the command and press Enter:

```
node asyncprg.js
```

Figure 7.3 displays the output of Code Snippet 1.

```
Topics to be discussed in this session are:
1.Asynchronous Programming
2.Synchronous Programming
Welcome to Node.js programming!

This session covers asynchronous and synchronous programming models.
```

Figure 7.3: Output of Code Snippet 1

Observe the program output. The file reading operation is running in the background. Therefore, the rest of the code after the callback function block is executed and the messages provided in the `console.log` functions are displayed first. Node.js does not stop the execution of the `console.log` functions even though the file reading operation is happening in the background. After the file reading operation is completed, contents of the text file are displayed.

- **Working of the callback Function in Code Snippet 1**

The request to read the contents of the text file is submitted to the operating system. On receiving the request, the operating system starts processing and sends the response. There should be an entity that can handle the response sent by the operating system and that entity is the callback function. The callback function is capable of handling the operating system's response.

When the request-response operations are in progress, the main thread of Node.js continues to execute other instructions in the program. Therefore, the `console.log` functions are executed, and the outputs are displayed.

- **Operational Flow of Code Snippet 1**

Figure 7.4 shows the visual representation of the operational flow of Code Snippet 1.

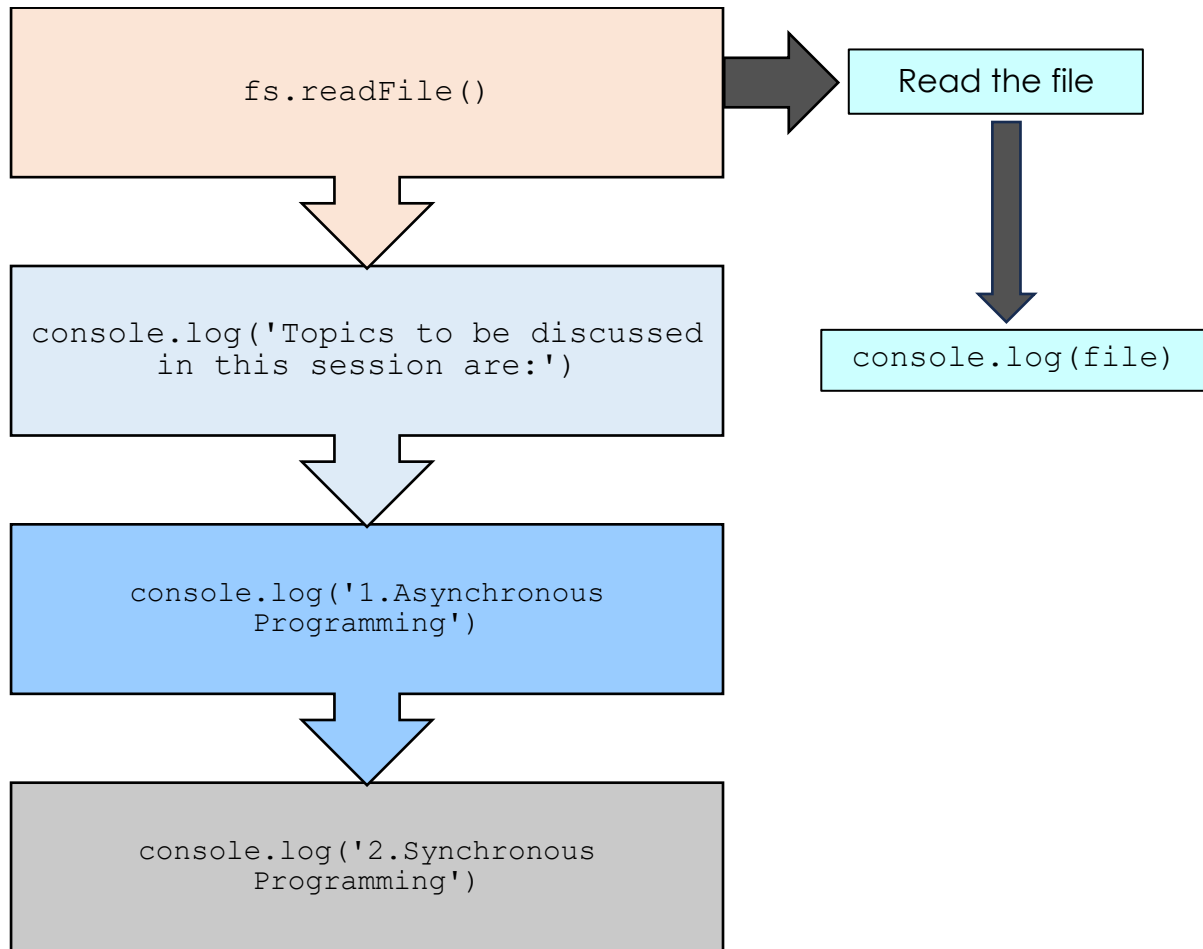


Figure 7.4: Operational Flow of Code in Code Snippet 1

7.4.2 Asynchronous Programming in Node.js Using Promise

An asynchronous Node.js program can have multiple or nested callback functions for simultaneous execution of tasks. This kind of arrangement, also known as a callback hell, in a program can lead to chaos and disorganization. A JavaScript object known as `Promise` solves the problem of disorganized callback functions in a program.

The object, `Promise`, handles the execution of multiple tasks in an organized manner in a program. A task can just be started, or already completed successfully, or rejected due to an error. Therefore, the `Promise` object has three states as shown in Figure 7.5.

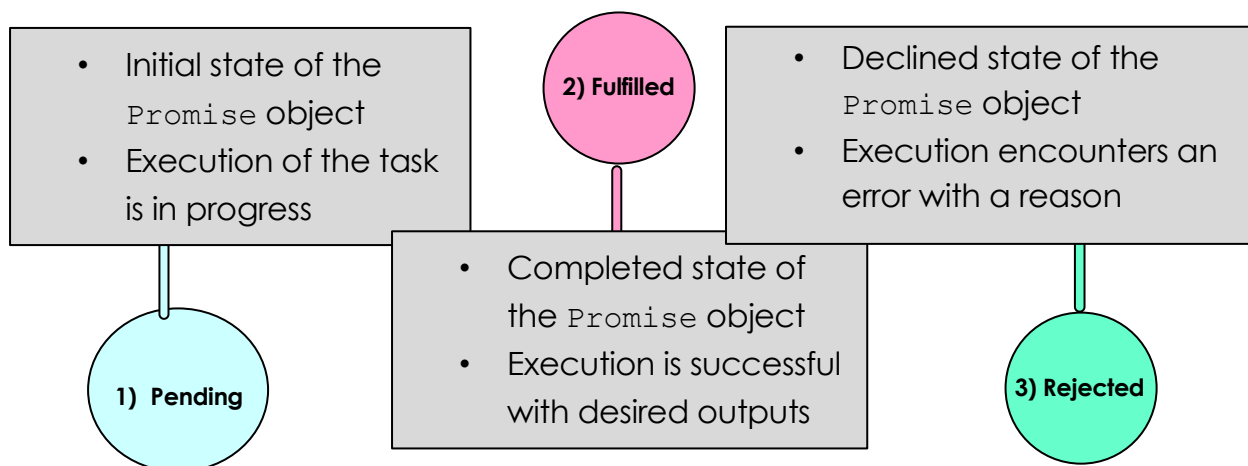


Figure 7.5: States of the Promise object

Consider an example code in Code Snippet 2 where the file reading operation is demonstrated using the `Promise` object.

Code Snippet 2:

```
const fs = require('fs');
const promise = new Promise(function (resolve, reject){
  fs.readFile('nodefile.txt','utf8', (err, data) => {
    if (err) {
      return reject (err)
    }
    resolve(data)
  })
});
promise
  .then ((data) => console.log(data))
  .catch((err) => console.log(err))
```

In Code Snippet 2:

- The `fs` module is used to handle the file read operation. The `Promise` constructor is used to create a new `Promise` object.
- The first argument to the constructor is a callback function. This callback function is called the `executor` function, which is to be executed by the constructor. The `executor` function accepts two functions as parameters.
- The first parameter is the `resolve` function, which creates the `Promise` object in a fulfilled or completed state. The second parameter is the `reject` function, which declines the creation of the `Promise` object with an error definition.
- The `fs.readFile` function reads the text in the `nodefile.txt` file asynchronously. The file reading operation takes place in the background while the remaining program tasks are being executed. If the

`nodefile.txt` file is available, the `Promise` object creation is successful, and the object moves to the resolved state. The contents of the text file is transmitted to the `.then` function and the output is displayed on the console.

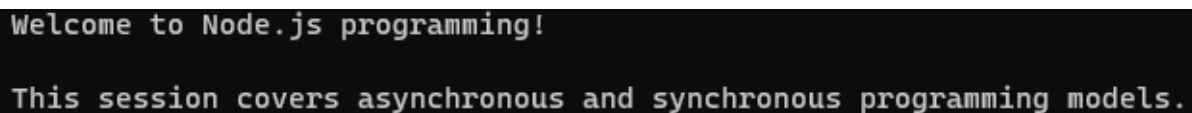
- If the `nodefile.txt` file is not available, the `Promise` object creation is unsuccessful and the object moves to the rejected state. An error is generated and handled by the `.catch` function and the error information is displayed on the console.

Save Code Snippet 2 with the file name, `asyncpromise.js`. Use the `nodefile.txt` file created earlier.

Type the command and press Enter:

```
node asyncpromise.js
```

Figure 7.6 displays the output of Code Snippet 2.



```
Welcome to Node.js programming!  
  
This session covers asynchronous and synchronous programming models.
```

Figure 7.6: Output of Code Snippet 2

Observe the program output. Since the text file is available, the program has not generated an error and the contents of the text file are displayed.

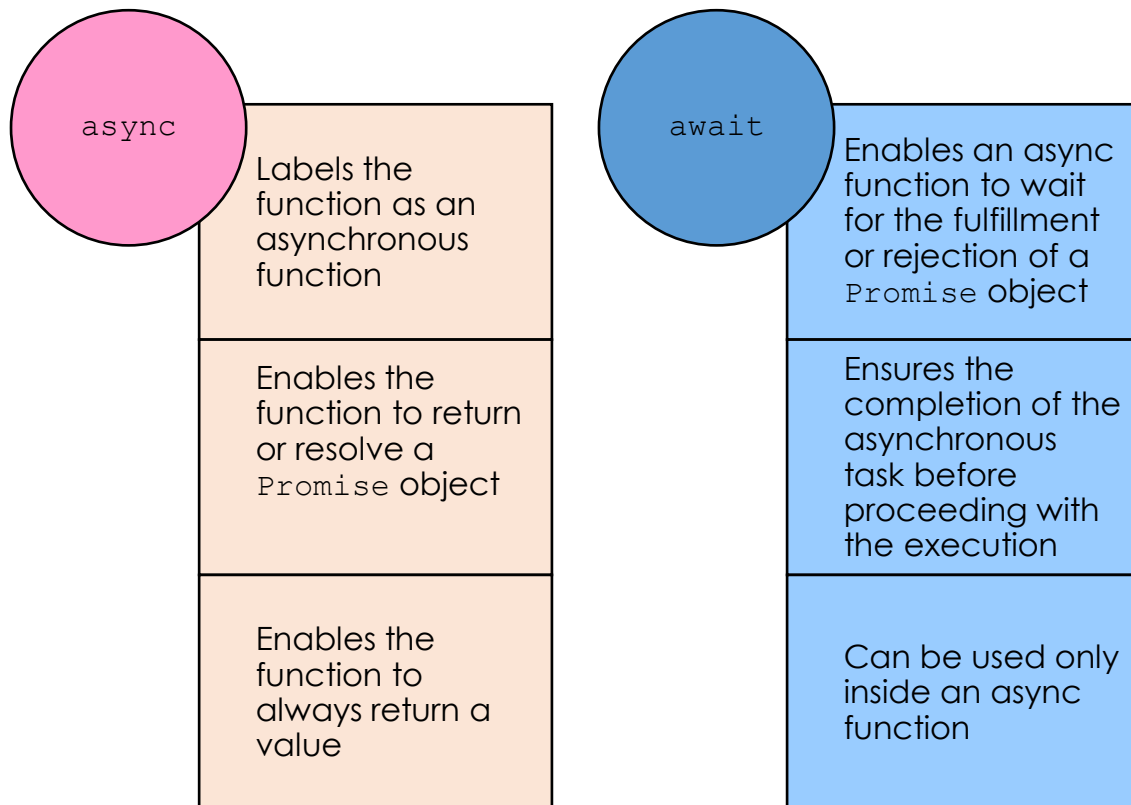
Delete the text file from the folder and run the program again. The program will throw a runtime error displaying the details of the error.

To understand the purpose of the promise object, consider the example of a restaurant. The chefs in the kitchen of the restaurant receive orders continuously for various dishes from the waiters. Here, the preparation of a dish is an asynchronous task. A waiter is a promise object who takes orders from the customers and passes them on to the chefs. Chefs are the asynchronous processors who execute the asynchronous task of preparing the dish thereby fulfilling the promises. Why is this whole process asynchronous? Multiple orders will be received and processed by multiple chefs simultaneously without waiting for the completion of any one particular task. In case a certain dish is not available, the waiter obtains an updated order from the customer. Thus, a promise is either fulfilled or rejected to create another promise.

To conclude, the `Promise` object handles both successful and unsuccessful asynchronous operations in a program.

7.4.3 Asynchronous Programming in Node.js Using `async/await`

The keywords `async` and `await` act as an added advantage for callback function and `Promise` object in Node.js. These keywords make an asynchronous program appear similar to a synchronous program thereby facilitating clarity in the sequence of events.



Consider the example code in Code Snippet 3 where the file reading operation is demonstrated using the `async` and `await` keywords.

Code Snippet 3:

```
const fs = require('fs');
const filepromise = function () {
  const promise = new Promise(function (resolve, reject){
    fs.readFile('nodefile.txt','utf8', (err, data) => {
      if (err) {
        return reject (err)
      }
      resolve(data)
    })
  });
  return promise;
};
async function readFileAsync() {
  try {
    const data = await filepromise();
```

```
console.log('File content:');  
console.log(data);  
} catch (error) {  
  console.error('Error:', error.message);  
}  
}  
readFileAsync();
```

In the example code, the `fs` module is used to handle the file read operation. The `Promise` constructor is used to create a new `Promise` object.

An asynchronous function with the name `readFileAsync` is declared using the `async` keyword. The `await` keyword is used inside the `async` function. The keyword pauses the execution of the `async` function until the fulfillment of the `Promise` object. The function `filepromise` returns a `Promise` object.

If the program is able to read the text file successfully, the `Promise` object is resolved, and the contents of the text file are displayed as the output. The `try` block handles the successful execution of the function. If the text file is not available, an error is generated at runtime, and the `catch` block handles the error.

Either the `try` block or the `catch` block is executed depending on the fulfillment or rejection of the `Promise` object. The code initiates the file read operation by invoking the `readFileAsync` function.

Save Code Snippet 3 with the file name, `fileawait.js`. Type the command and press Enter:

```
node fileawait.js
```

Figure 7.7 displays the output of Code Snippet 3.

```
File content:  
Welcome to Node.js programming!  
  
This session covers asynchronous and synchronous programming models.
```

Figure 7.7: Output of Code Snippet 3

Observe the program output. The `Promise` object is successfully resolved, the `try` block is executed, and the text file contents are displayed on the console.

7.5 Synchronous Programming in Node.js

A synchronous Node.js program executes the instructions in a sequential way. This approach can lead to blockade of critical instructions and make the program unresponsive.

Consider an example code in Code Snippet 4 where the text file reading operation is demonstrated using the synchronous programming model.

Code Snippet 4:

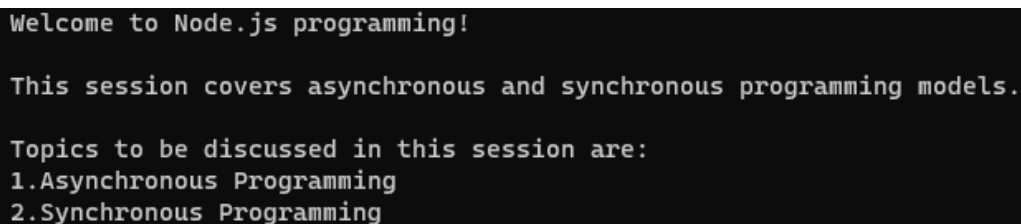
```
const fs = require('fs');
const data = fs.readFileSync('nodefile.txt', 'utf8');
console.log(data);
console.log('Topics to be discussed in this session are:');
console.log('1.Asynchronous Programming');
console.log('2.Synchronous Programming');
```

In the example code, the `fs` module is used to perform the file read operation. The synchronous function `fs.readFileSync` reads the contents of the text file, `nodefile.txt`, which is passed as the function argument. The program is executed sequentially. Therefore, the `console.log` statements are executed only after the function `fs.readFileSync` completes the file reading operation.

Save Code Snippet 4 with the file name `syncprg.js`. Type the command and press Enter:

```
node syncprg.js
```

Figure 7.8 displays the output of Code Snippet 4.



```
Welcome to Node.js programming!

This session covers asynchronous and synchronous programming models.

Topics to be discussed in this session are:
1.Asynchronous Programming
2.Synchronous Programming
```

Figure 7.8: Output of Code Snippet 4

Observe the program output. The contents of the text file are displayed first and then the messages provided in the `console.log` functions are displayed.

Operational Flow of Code Snippet 4

Figure 7.9 shows the visual representation of the operational flow of Code Snippet 4.

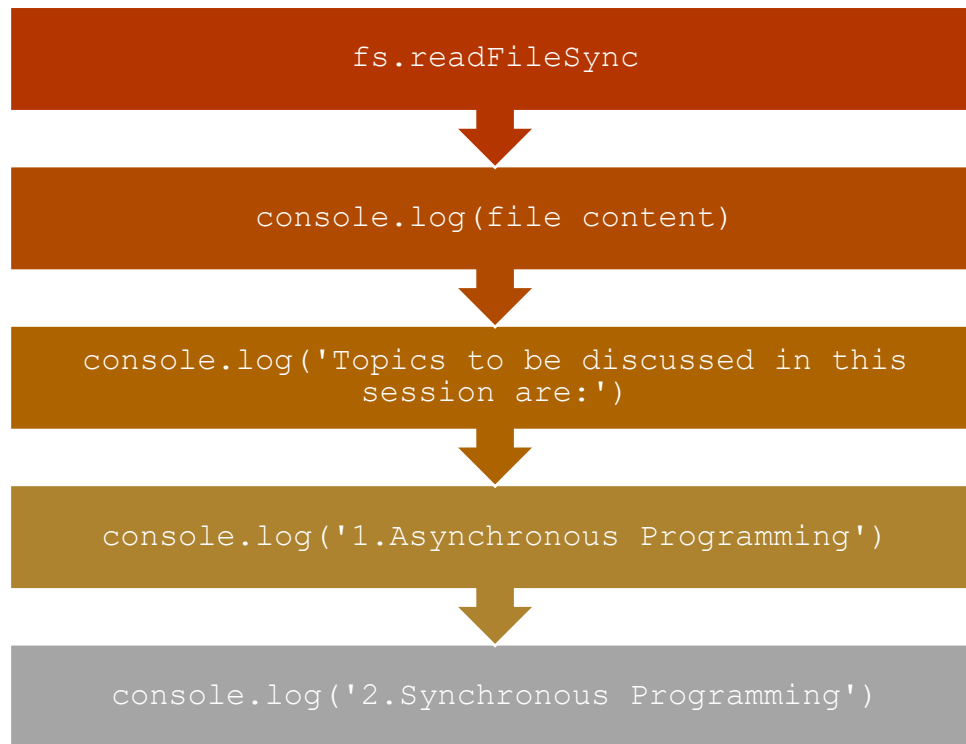


Figure 7.9: Operational Flow of Code in Code Snippet 4

7.6 Introduction to Event Loop in Node.js

The asynchronous programming model of Node.js is also known as the Input/Output (I/O) non-blocking model. An event loop manages the asynchronous and non-blocking operations in Node.js. The event loop keeps on running continuously until there are asynchronous operations in the program.

7.6.1 Working of an Event Loop

Figure 7.10 shows a diagrammatic representation of the working of an event loop on the Node.js server.

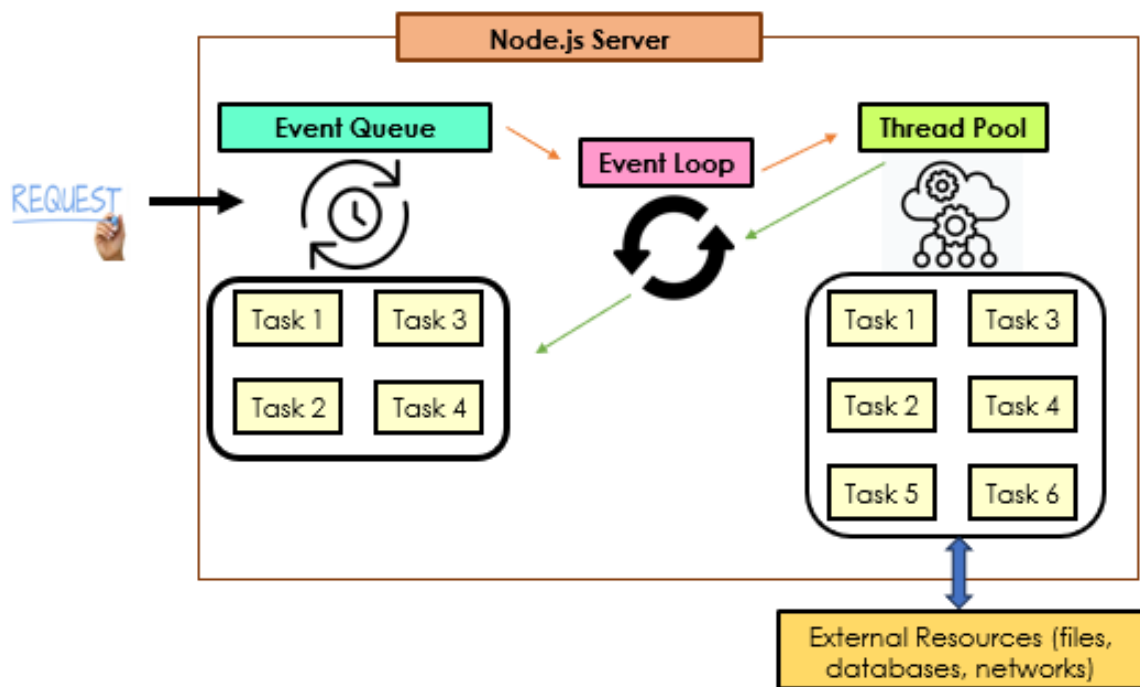
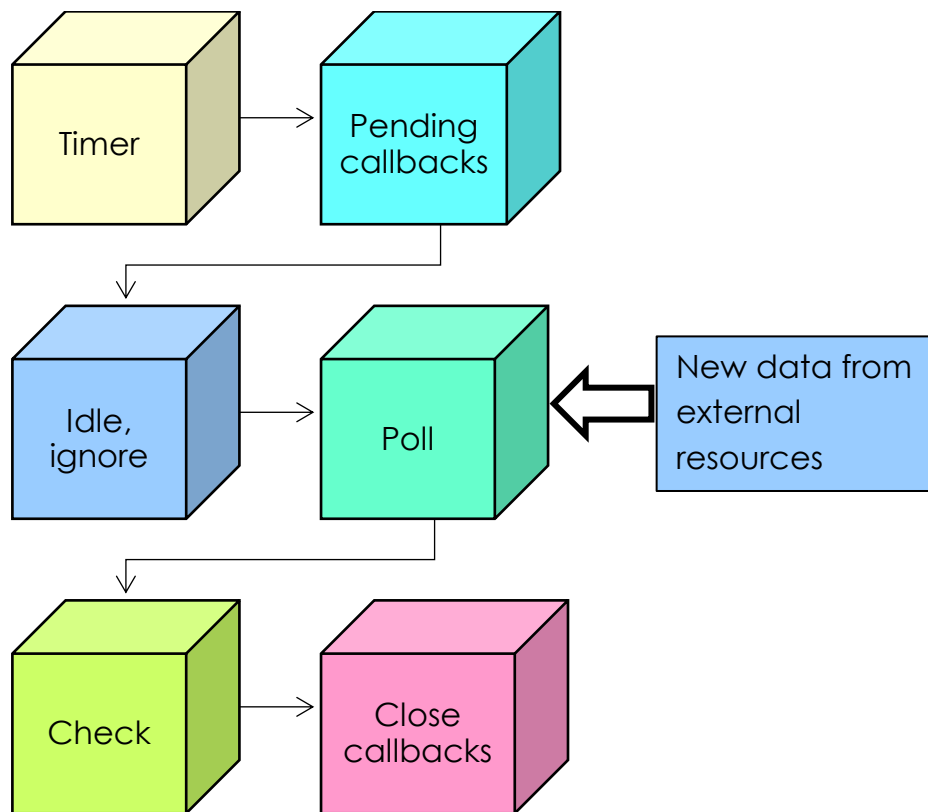


Figure 7.10: Working of Event Loop

The components of Node.js server that are responsible for the looping the events are as follows:

- **Event queue:** This component is also known as the callback queue or message queue. Event queue is a data structure that stores asynchronous tasks or callbacks. The event loop picks up the task from the event queue for execution.
- **Event loop:** This loop is the most important component of Node.js server. The loop keeps on checking the event queue for any tasks to execute. The event loop consists of six phases. Each phase has a set of tasks to perform.

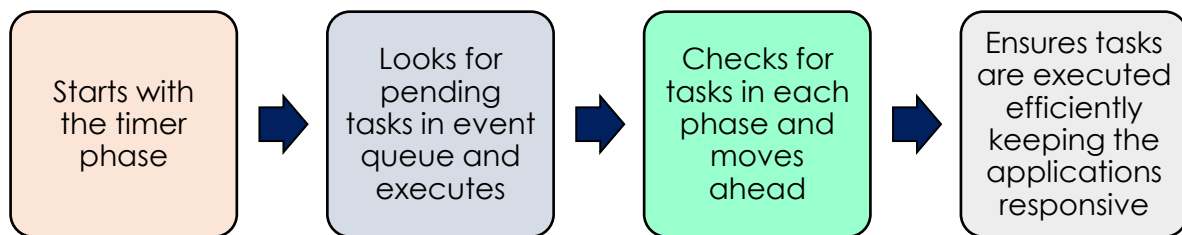


Event loop phases are as follows:

- **Timer:** This phase checks for any timers that have attained their specified duration. The timer module is a part of the Node.js framework. The timer module executes callback functions according to the specified intervals. There are two global functions, such as `setTimeout` and `setInterval`, in the timer module. The `setTimeout` function facilitates the setting up of a timer that can execute a specific task upon the expiry of the timer. The `setInterval` function helps to execute the tasks repeatedly, after a specific interval.
- **Pending callbacks:** In this phase, the pending callback functions, related to input/output, file handling, and so on, are executed from the event queue.
- **Idle/ignore:** This phase is usually dedicated to the not-so-important tasks in the event loop. Internal purpose tasks or background tasks such as garbage collection are performed in the idle phase.
- **Poll:** This phase is entrusted with the task of checking new input/output events and processes.
- **Check:** This phase starts after the poll phase becomes idle. The `setImmediate` function, a timer, executes the callback functions in the check phase.

- **Close callbacks:** This phase executes callback functions that are primarily involved in the clean-up tasks such as closing the database connections or terminating the server connections.
- **Thread pool:** The thread pool is in charge of executing the asynchronous operations. The Node.js server informs the thread pool of an asynchronous task. The asynchronous task runs as a separate thread in the thread pool. Concurrently, the main operations run as a separate thread.

To conclude, the event loop:



7.6.2 Event Loop Example

Consider an example code for an event loop in Code Snippet 5.

Code Snippet 5:

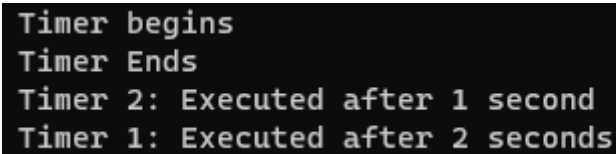
```
console.log('Timer begins');
setTimeout(() => {
  console.log('Timer 1: Executed after 2 seconds');
}, 2000);
setTimeout(() => {
  console.log('Timer 2: Executed after 1 second');
}, 1000);
console.log('Timer Ends');
```

In the example code, two timers are set using the `setTimeout` function. The first timer, Timer 1, will execute after two seconds. The second timer, Timer 2, will execute after one second. The tasks are executed asynchronously. Therefore, timers run in the background and are added to the event queue when they are ready for execution. The event loop checks the event queue and picks up the callback functions for Timer 1 and Timer 2, respectively.

Save Code Snippet 5 with the file name, `eventloopprg.js`. Type the command and press Enter:

```
node eventloopprg.js
```

Figure 7.11 displays the output of Code Snippet 5.

A terminal window with a black background and white text showing the output of a Node.js program. The output consists of four lines: 'Timer begins', 'Timer Ends', 'Timer 2: Executed after 1 second', and 'Timer 1: Executed after 2 seconds'.

```
Timer begins  
Timer Ends  
Timer 2: Executed after 1 second  
Timer 1: Executed after 2 seconds
```

Figure 7.11: Output of Code Snippet 5

Observe the program output. The first `console.log` function is executed. Meanwhile, the callback functions are running in the background. Therefore, Node.js continues to execute the second `console.log` function. It then, executes Timer 2 followed by Timer 1.

7.7 Summary

- In an asynchronous programming model, tasks are executed simultaneously.
- The synchronous programming model follows the sequential execution of tasks.
- Node.js relatively follows the asynchronous programming model with the help of a callback function, `Promise` object, and `async/await` keywords.
- The callback function executes the asynchronous tasks in the background while other tasks are executed without any blockade.
- The `Promise` object acts as a complementary addition to the callback function.
- The `async` keyword ensures that the `async` function resolves a `Promise` object.
- The `await` keyword should be used inside an `async` function only.
- The event loop architecture in Node.js consists of six phases such as timer, pending callbacks, idle, ignore, poll, check, and close callbacks.
- An event loop maintains the responsiveness of the applications ensuring efficient execution of asynchronous tasks.

Test Your Knowledge

1. Which approach do the applications using synchronous programming follow?
 - a) Bottom-up Approach
 - b) Top-down Approach
 - c) Routing Approach
 - d) Sequential Approach
2. In asynchronous programming, how is a program executed?
 - a) Sequentially
 - b) Concurrently
 - c) Multiple tasks executing at once
 - d) None of the these
3. Which of the following functions can be used to perform asynchronous programming in Node.js applications?
 - a) `reverse` Function
 - b) `callback` Function
 - c) `addition` Function
 - d) `Promise` Function
4. Which of the following is not a state of the `Promise` object?
 - a) Fulfilled
 - b) Rejected
 - c) Pending
 - d) Stop

5. Add the missing line in the given code:

```
const fs = require('fs');  
// Add missing line here  
if (err) {  
  console.error(err);  
  return;  
}  
console.log(data);  
});  
console.log('You have started working on asynchronous  
programming.');
```

a) fs.readFile('program.txt', 'utf8', (err, data) => {
b) fs.readFile('program.txt', 'utf11', (err) =>
c) fs.readFile('program.txt', 'utf8', (data) =>
d) fs.readFile('program.txt', 'utf8' =>

Answers to Test Your Knowledge

1	b
2	a
3	b, d
4	d
5	a

Try It Yourself

1. Create a Node.js application to read a text file.
 - a) Create a text file using Notepad. Enter the given content in the read Text file:
 - i. Now, you have started working on a Node.js application.
 - ii. You have learned how to perform asynchronous and synchronous programming.
 - b) Save the text file with the name, `readTextFile.txt` in a particular folder on the local system.
 - c) Save the code with the file name `program.js`.
 - d) Display the result in the console.
2. Create a Node.js application to read a file using the `async` and `await` functions.
 - a) Create a text file using Notepad. Enter the given content in the text file:
 - i. Mangos are Yellow.
 - ii. Strawberries are Red.
 - iii. Raspberries are Red.
 - b) Save the text file as `fruit.txt` on the local system.
 - c) Define a function that will return `Promise`.
 - d) Create an asynchronous function for reading file.
 - e) Save the code as `programAsync.js`.
 - f) Display the result in the console.
3. Create a simple program and display the given statement in the console using the timer module. Display Statement 1 after 3 seconds and Statement 2 after 4 seconds.

Statement 1: Fruits are beneficial for your health.
Statement 2: Fruits are low in calories.